

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт по лабораторным работам №1–5

По дисциплине: «Теория графов»

На тему: «Алгоритмы на графах»

Вариант №44

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Содержание

Введение	3
1 Постановка задачи	4
2 Математическое описание	7
2.1 Генерация графа и основные понятия	7
2.2 Связность и точки сочленения	12
2.3 Максимальный поток и поток минимальной стоимости . .	16
2.4 Остовы, код Прюфера и независимые множества рёбер . .	19
2.5 Эйлеровы циклы и фундаментальная система разрезов . .	22
3 Результаты работы	23

Введение

Теория графов — раздел дискретной математики, изучающий объекты и связи между ними с помощью графов, состоящих из вершин и рёбер. В рамках теории графов рассматриваются задачи анализа связности, поиска путей, построения остовов, исследования циклов, потоков и паросочетаний. Графовые модели применяются при описании транспортных и компьютерных сетей, социальных связей, структур данных и других сложных систем.

В отчёте представлено описание выполнения цикла лабораторных работ по дисциплине «Теория графов». Целью работ является изучение способов представления графов и практическая реализация алгоритмов для их генерации, анализа и преобразования.

Разработанная консольная программа на языке C++ объединяет результаты пяти лабораторных работ. Программа выполняет случайную генерацию графа и весовых матриц, вычисляет эксцентриситеты вершин, выделяет центр графа и диаметральные вершины, ищет маршруты, кратчайшие пути, точки сочленения и потоки, строит минимальный остов, кодирует его кодом Прюфера, находит максимальное независимое множество рёбер, строит эйлеров цикл и фундаментальную систему разрезов. Исходный граф, сформированный в первой работе, затем используется в последующих работах.

В отчёте рассмотрены математическое описание использованных алгоритмов, особенности их программной реализации и результаты работы программы на сгенерированных графах.

1 Постановка задачи

Цель работы: разработать единый программный комплекс с текстовым пользовательским интерфейсом для генерации, анализа и преобразования графов, объединяющий алгоритмы теории графов, реализованные в ходе выполнения пяти лабораторных работ.

Задачи.

Первая лабораторная работа:

1. сформировать случайным образом связный ациклический ориентированный граф в соответствии со смещённым распределением Вейбулла с заданным пользователем количеством вершин;
2. реализовать генерацию весовой матрицы в соответствии со смещённым распределением Вейбулла с положительными, отрицательными или смешанными весами при заданном пользователем количестве рёбер;
3. посчитать эксцентриситеты вершин;
4. выделить центр графа;
5. определить диаметральные вершины;
6. реализовать метод Шимбелла для сгенерированной весовой матрицы при заданном пользователем количестве рёбер;
7. выводить максимальный или минимальный путь, полученный методом Шимбелла, в виде весовой матрицы;
8. определить возможность построения маршрута между двумя заданными пользователем вершинами и посчитать количество таких маршрутов;
9. реализовать пользовательское меню;

Вторая лабораторная работа:

10. найти точки сочленения в сгенерированном графе;
11. найти кратчайший путь между выбранными вершинами с помощью алгоритма Дейкстры для отрицательных весов, вывести расстояние, вектор расстояний и последовательность вершин пути;

12. сравнить скорость работы алгоритмов по количеству итераций;

Третья лабораторная работа:

13. сформировать случайным образом матрицу пропускных способностей в соответствии со смещённым распределением Вейбулла на основе сгенерированного ориентированного графа;
14. сформировать случайным образом матрицу стоимостей в соответствии со смещённым распределением Вейбулла на основе сгенерированного ориентированного графа;
15. найти максимальный поток для полученной матрицы пропускных способностей по алгоритму Форда–Фалкерсона;
16. вычислить поток минимальной стоимости величины $\lfloor 2/3 \cdot \max \rfloor$, здесь $\max$ — найденный максимальный поток;

Четвёртая лабораторная работа:

17. найти число остовных деревьев неориентированного графа с помощью матричной теоремы Кирхгофа;
18. построить минимальный по весу остов для неориентированного взвешенного графа алгоритмом Краскала;
19. закодировать остов кодом Прюфера с сохранением весов рёбер и декодировать его;
20. найти максимальное независимое множество рёбер на исходном графе и полученном остове по выбору пользователя;

Пятая лабораторная работа:

21. проверить, является ли неориентированный граф эйлеровым;
22. при необходимости модифицировать граф с сохранением связности и вывести сведения о выполненных изменениях;
23. построить эйлеров цикл;
24. построить минимальный остов алгоритмом Краскала;
25. получить фундаментальную систему разрезов и вывести её на экран;

26. получить остальные разрезы из фундаментальных с помощью операции симметрической разности для выбранных пользователем разрезов.

2 Математическое описание

2.1 Генерация графа и основные понятия

Графом $G(V, E)$ называется совокупность двух множеств:

- непустое множество V — множество вершин;
- E , множество двуэлементных подмножеств множества V , — множество рёбер.

Ребро можно трактовать не только как множество $\{v_1, v_2\}$, но и как пару вершин (v_1, v_2) .

$$p \stackrel{\text{Def}}{=} p(G) \stackrel{\text{Def}}{=} |V|, \quad q \stackrel{\text{Def}}{=} q(G) \stackrel{\text{Def}}{=} |E|.$$

Если рёбра задаются упорядоченными парами, то граф называется ориентированным. В противном случае — неориентированным. Степенью вершины $d(v)$ называется количество рёбер, инцидентных этой вершине.

Для неориентированного графа справедлива лемма о рукопожатиях. Сумма степеней вершин графа равна удвоенному количеству рёбер.

$$\sum_{v \in V} d(v) = 2q.$$

Маршрутом в графе называется чередующаяся последовательность вершин и рёбер, начинающаяся и заканчивающаяся вершиной, в которой любые два соседних элемента инцидентны.

- Если все рёбра различны, то маршрут называется цепью.
- Если все вершины различны, то маршрут называется простой цепью.

Если $v_0 = V_k$, то маршрут замкнут, иначе — открыт.

Цепь, соединяющая вершины u и v , обозначается $\langle u, v \rangle$. Говорят, что две вершины в графе связны, если существует соединяющая их цепь. *Граф, в котором все вершины связны, называется связным.*

Замкнутая цепь называется циклом, замкнутая простая цепь называется простым циклом. Число циклов в графе обозначается $z(G)$. *Граф без циклов называется ациклическим.*

В первой лабораторной работе связный ациклический граф генерируется случайным образом на основе смещённого распределения Вейбулла. Распределение задаётся функцией, приведённой в справочнике Р. Н. Вадзинского:

$$F(y) = 1 - \exp \left(- \left(\frac{y - y_0}{a} \right)^c \right),$$

где y — случайная величина, a — параметр масштаба, c — параметр формы ($c > 0$), y_0 — параметр смещения.

Для генерации случайных чисел по смещённому распределению Вейбулла используется формула

$$y = y_0 + a(-\ln r)^{1/c}.$$

Здесь r — равномерно распределённая на интервале $(0, 1)$ случайная величина.

На рис. 1 показана функция риска распределения Вейбулла при параметрах $a = 1$, $c = 0,5; 1,5; 2,5$, $y_0 = 0$.

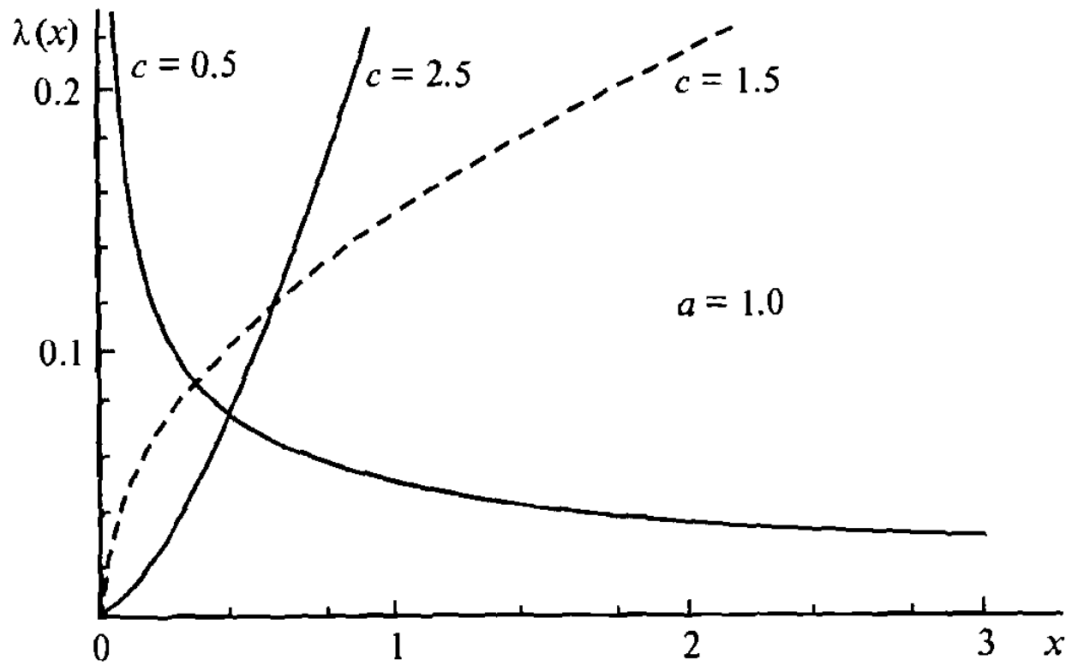


Рис. 1: Функция риска распределения Вейбулла

На рис. 2 приведены гистограммы 1000 сгенерированных значений для тех же параметров. При $c < 1$ большая часть значений сосредоточена возле нуля. При увеличении c значения становятся более сконцентрированными, а максимум гистограммы смещается вправо.

Смещённое распределение Вейбулла при различных значениях c

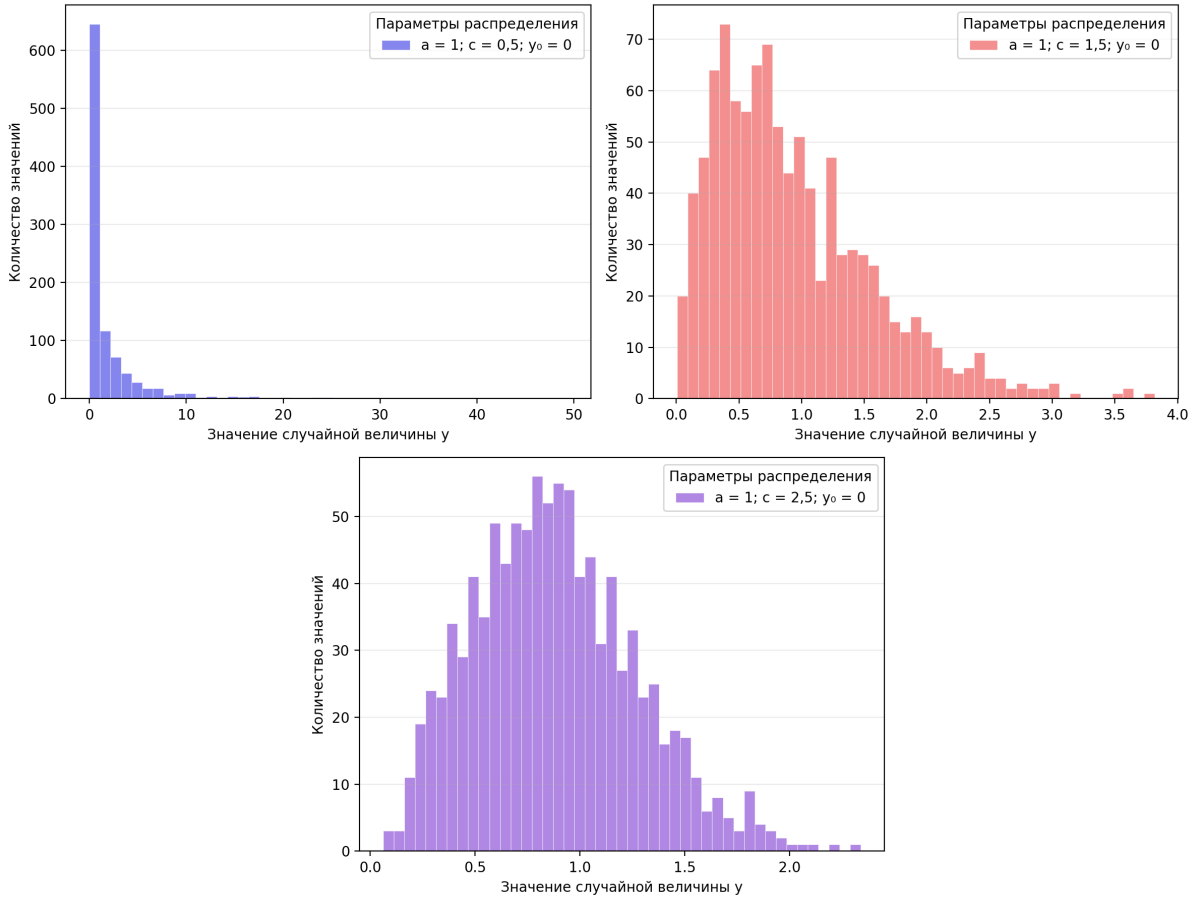


Рис. 2: Гистограммы смещённого распределения Вейбулла при различных значениях c

Для генерации матрицы смежности использовались параметры $a = 0.17$, $c = 2.0$, $y_0 = 0.05$. При формировании графа значение степени i -й вершины вычисляется по формуле

$$d_i = \text{round}((p - 1)y_i), \quad i = 1, \dots, p,$$

где p — количество вершин графа, y_i — значение случайной величины, полученное по смещённому распределению Вейбулла. Для графа из 10 вершин формула принимает вид

$$d_i = \text{round}(9y_i).$$

Граф строится по полученной последовательности степеней вершин при выполнении условия

$$1 \leq d_i \leq p - 1 \quad \text{для всех } i = 1, \dots, p.$$

Если хотя бы одна степень не удовлетворяет этому условию, последовательность генерируется заново. Такой набор параметров подобран

экспериментально. Он обеспечивает умеренные степени вершин и не приводит к формированию почти полного графа.

Гистограмма 1000 значений распределения с выбранными параметрами приведена на рис. 3. На том же рисунке показаны рассчитанные значения степеней для графа из 10 вершин.

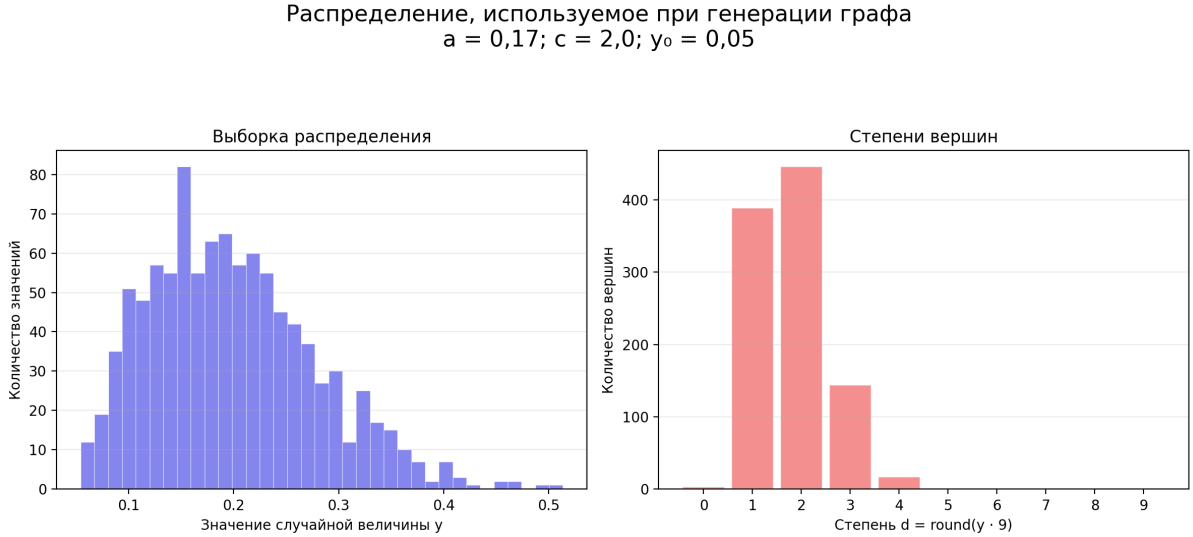


Рис. 3: Распределение Вейбулла и значения степеней при $p = 10$

В рамках работы смещённое распределение Вейбулла используется также для генерации весов рёбер и сети потоков. Для весовой матрицы пользователь может выбрать знак генерируемых весов: положительные, отрицательные или смешанные. При генерации со смешанными весами знак выбирается случайно.

Расстоянием между вершинами u и v (обозначается $d(u, v)$) называется длина кратчайшей цепи $\langle u, v \rangle$, а сама кратчайшая цепь называется геодезической.

$$d(u, v) \stackrel{\text{Def}}{=} \min_{\langle u, v \rangle} \langle u, v \rangle.$$

$$\text{Если } \nexists \langle u, v \rangle, \text{ то } d(u, v) \stackrel{\text{Def}}{=} +\infty.$$

Диаметром графа G называется длиннейшая геодезическая. Длина диаметра обозначается $D(G)$:

$$D(G) \stackrel{\text{Def}}{=} \max_{u, v \in V} d(u, v).$$

Эксцентриситетом $e(v)$ вершины v в связном графе $G(V, E)$ называется максимальное расстояние от v до любой другой вершины графа G :

$$e(v) \stackrel{\text{Def}}{=} \max_{u \in V} d(u, v).$$

Радиусом $R(G)$ графа G называется наименьший из эксцентриситетов вершин:

$$R(G) \stackrel{\text{Def}}{=} \min_{v \in V} e(v).$$

Вершина v называется центральной, если её эксцентриситет совпадает с радиусом графа, т.е. $e(v) = R(G)$.

Множество центральных вершин называется центром графа и обозначается $C(G)$:

$$C(G) \stackrel{\text{Def}}{=} \{v \in V \mid e(v) = R(G)\}.$$

Метод Шимбелла использует степени весовой матрицы для нахождения весов маршрутов с фиксированным количеством рёбер. В данной работе для каждой пары вершин определяется минимальный или максимальный суммарный вес маршрута, содержащего ровно k рёбер.

Пусть $W = (w_{ij})$ — весовая матрица графа. При Шимбелловом умножении произведение элементов заменяется сложением весов, а сложение полученных значений — выбором минимума или максимума. Правила выполнения операций приведены на рис. 4.

1. Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть

$$\begin{cases} a \cdot b = b \cdot a \Rightarrow a + b = b + a, \\ a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \end{cases} \quad (3.8.1)$$

2. Операция сложения двух величин a и b заменяется выбором из этих величин минимального (максимального) элемента, то есть

$$a + b = b + a = \min(\max) \{a, b\} \quad (3.8.2)$$

нули при этом игнорируются. Минимальный или максимальный элемент выбирается из ненулевых элементов. Нуль в результате операции (3.8.2) может быть получен лишь тогда, когда все элементы из выбираемых — нулевые.

$$\omega_{ij}^{(2)} = \min(\max) \{(\omega_{i1}^{(1)} + \omega_{1j}^{(1)}), (\omega_{i2}^{(1)} + \omega_{2j}^{(1)}), \dots, (\omega_{in}^{(1)} + \omega_{nj}^{(1)})\}$$

Рис. 4: Матричные операции метода Шимбелла

Для произвольного количества рёбер вычисления продолжаютс я рекуррентно. Произведение матриц в следующей формуле является Шимбелловым умножением и выполняется по правилам, приведённым на рисунке 4:

$$W^{(1)} = W, \quad W^{(k)} = W^{(k-1)} \cdot W.$$

Следовательно, элемент $w_{ij}^{(k)}$ результирующей матрицы содержит минимальный или максимальный вес маршрута из v_i в v_j , состоящего ровно из k рёбер. При $k = 0$ диагональные элементы матрицы равны нулю, а остальные элементы соответствуют отсутствующим маршрутам.

2.2 Связность и точки сочленения

Отношение связности вершин называется эквивалентностью. *Классы эквивалентности по отношению связности называются компонентами связности графа.*

Вершина графа называется точкой сочленения, если её удаление увеличивает число компонент связности. Мостом называется ребро, удаление которого увеличивает число компонент связности.

Для поиска точек сочленения граф обходится в глубину. Для каждой вершины определяется, существует ли из её поддерева путь к ранее посещённым вершинам в обход неё. Если такого пути нет, удаление этой вершины разделит граф, поэтому она является точкой сочленения. Для корневой вершины отдельно проверяется количество независимых ветвей обхода. В реализации ориентированный граф рассматривается как неориентированный.

Псевдокод обхода в глубину с вычислением нижних меток приведён на рис. 5.

```

1.  procedure BiComp(v);
2.  begin
3.    num[v] := i; L[v] := i; i := i + 1;
4.    for u ∈ list[v] do
5.      if num[u] = 0 then
6.        begin
7.          SE ← vu; father[u] := v; BiComp(u)
8.          L[v] := min(L[v], L[u]);
9.          if L[u] ≥ num[v] then
10.           {получить новый блок; для этого из
               стека SE надо вытолкнуть все ребра
11.         end
12.       else if num[u] < num[v] and u ≠ father[v] then
13.         begin
14.           SE ← vu; L[v] := min(L[v], num[u]);
15.         end
16.     end;
17.   begin
18.     i := 1; SE := nil; father[v0] := ∅;
19.     for v ∈ V do num[v] := 0;
20.     BiComp(v0)
21.   end.

```

Рис. 5: Псевдокод поиска точек сочленения

Стандартный алгоритм Дейкстры находит кратчайшие пути от начальной вершины s до остальных вершин графа с неотрицательными весами. На каждой итерации выбирается вершина с минимальным текущим весом пути. Затем веса путей до соседних вершин пересчитываются в соответствии с неравенством треугольника. После этого выбранная вершина считается обработанной окончательно. Псевдокод стандартного алгоритма приведён на рис. 6.

Вход: орграф $G(V, E)$, заданный матрицей длин дуг $C : \text{array}[1..p, 1..p] \text{ of real}$; s и t — вершины графа.

Выход: векторы $T : \text{array}[1..p] \text{ of real}$; и $H : \text{array}[1..p] \text{ of } 0..p$. Если вершина v лежит на кратчайшем пути от s к t , то $T[v]$ — длина кратчайшего пути от s к v ; $H[v]$ — вершина, непосредственно предшествующая v на кратчайшем пути.

```

for  $v$  from 1 to  $p$  do
   $T[v] := \infty$  { кратчайший путь неизвестен }
   $X[v] := 0$  { все вершины не отмечены }
end for
 $H[s] := 0$  {  $s$  ничего не предшествует }
 $T[s] := 0$  { кратчайший путь имеет длину 0... }
 $X[s] := 1$  { ... и он известен }
 $v := s$  { текущая вершина }
 $M := \emptyset$  { обновление пометок }
for  $u \in \Gamma(v)$  do
  if  $X[u] = 0$  &  $T[u] > T[v] + C[v, u]$  then
     $T[u] := T[v] + C[v, u]$  { найден более короткий путь из  $s$  в  $u$  через  $v$  }
     $H[u] := v$  { запоминаем его }
  end if
end for
 $m := \infty$ ;  $v := 0$ 
{ поиск конца кратчайшего пути }
for  $u$  from 1 to  $p$  do
  if  $X[u] = 0$  &  $T[u] < m$  then
     $v := u$ ;  $m := T[u]$  { вершина  $v$  заканчивает кратчайший путь из  $s$  }
  end if
end for
if  $v = 0$  then
  stop { пет пути из  $s$  в  $t$  }
end if
if  $v = t$  then
  stop { найден кратчайший путь из  $s$  в  $t$  }
end if
 $X[v] := 1$  { найден кратчайший путь из  $s$  в  $v$  }
goto  $M$ 

```

Рис. 6: Псевдокод стандартного алгоритма Дейкстры

Для работы с отрицательными весами алгоритм модифицирован: вершина не фиксируется окончательно и при нахождении пути меньшего веса повторно помещается в очередь с приоритетами. Поэтому её вес может уменьшаться несколько раз. Сравнение стандартного и модифицированного алгоритмов приведено на рис. 7.

- Для графов с отрицательными весами (без циклов с суммарным отрицательным весом) время выполнения алгоритма экспоненциальное.

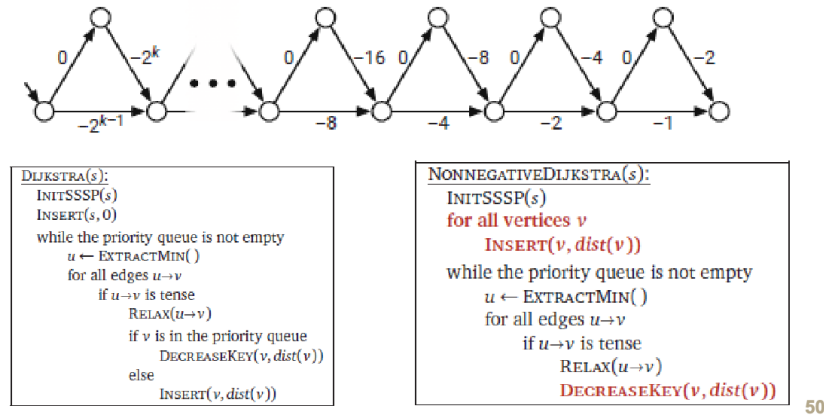


Рис. 7: Сравнение алгоритмов для неотрицательных и отрицательных весов

Для защиты от закливания алгоритм подсчитывает количество уменьшений веса каждой вершины. Если для некоторой вершины оно достигает количества вершин графа p , выполнение прекращается и фиксируется наличие отрицательного цикла. При его отсутствии алгоритм завершается после опустошения очереди. Пример изменения весов приведён на рис. 8 и 9.

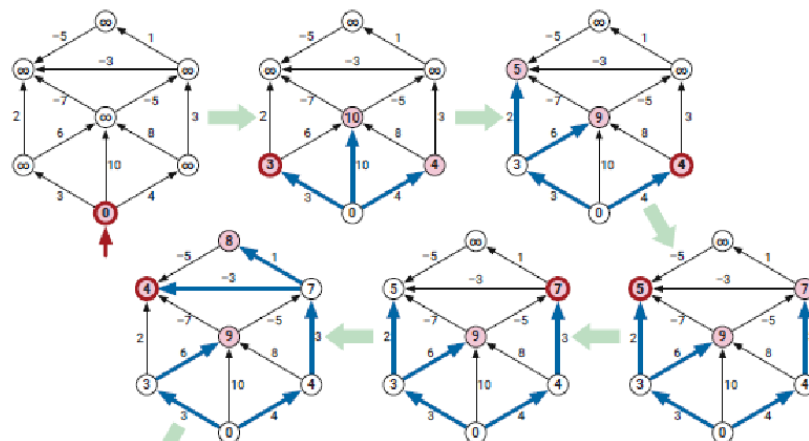


Рис. 8: Последовательное изменение весов при работе алгоритма, часть 1

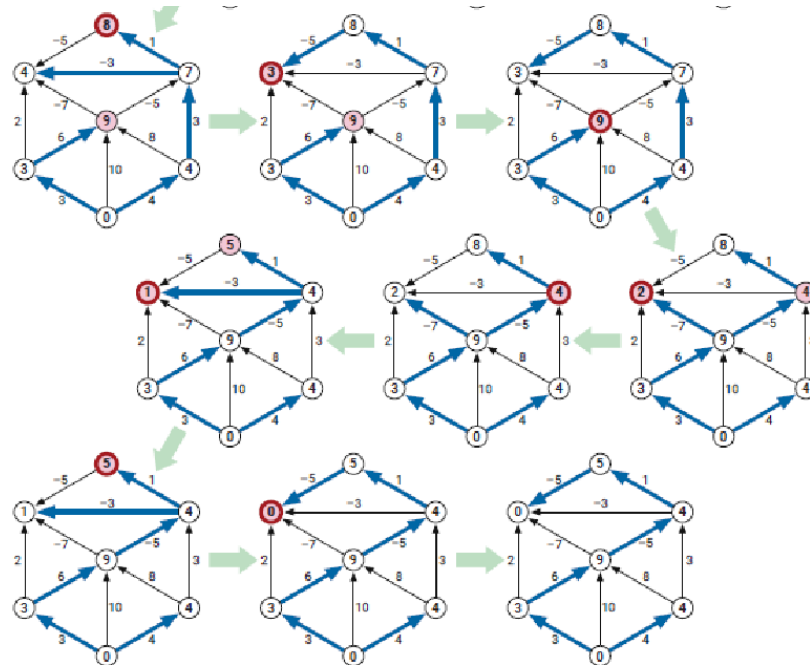


Рис. 9: Последовательное изменение весов при работе алгоритма, часть 2

Результаты сравнения алгоритмов по количеству итераций и соответствующие графики приведены в [разделе «Результаты работы»](#). По результатам запусков поиск точек сочленения оказался эффективнее модифицированного алгоритма Дейкстры.

2.3 Максимальный поток и поток минимальной стоимости

Пусть $G = (V, E)$ — орграф с одним истоком $s \in V$ и одним стоком $t \in V$, где $s \neq t$.

Сетью $S = (G; c)$ называется орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное число $c(e)$, называемое пропускной способностью этой дуги e .

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое потоком по этой дуге e .

Для поиска максимального потока применяется алгоритм Форда — Фалкерсона. В данной работе он реализован с помощью расстановки меток. Сначала поток по всем дугам полагается равным нулю, а исток получает начальную метку. Далее метки распространяются по дугам, по которым поток можно увеличить, и по обратным дугам, по которым ранее проведённый поток можно уменьшить. Каждая метка хранит предыдущую вершину, направление изменения потока и наименьшую пропускную способность найденной цепи.

Если метка достигает стока, по сохранённым вершинам восстанавливается увеличивающая цепь и поток вдоль неё изменяется на найденную

величину. После этого метки расставляются заново. Когда сток больше нельзя пометить, увеличивающих цепей не остаётся и полученный поток является максимальным. Псевдокод алгоритма приведён на рис. 10 и 11.

Вход: сеть $G(V, E)$ с источником s и стоком t , заданная матрица пропускных способностей $C : \text{array } [1..p, 1..p] \text{ of real}$.
Выход: матрица максимального потока $F : \text{array } [1..p, 1..p] \text{ of real}$.
for $u, v \in V$ **do**
 $F[u, v] := 0$ { вначале поток нулевой }
end for
 $M : \{ \text{итерация увеличения потока} \}$
for $v \in V$ **do**
 $S[v] := 0; N[v] := 0; P[v] := (+, \text{nil}, 0)$ { инициализация }
end for
 $S[s] := 1; P[s] := (+, \text{nil}, \infty)$ { так как $s \in S$ }
repeat
 $a := 0$ { признак расширения S }
 for $v \in V$ **do**
 if $S[v] = 1 \ \& \ N[v] = 0$ **then**
 for $u \in \Gamma(v)$ **do**
 if $S[u] = 0 \ \& \ F[v, u] < C[v, u]$ **then**
 $S[u] := 1; P[u] := (+, v, \min(P[v].\delta, C[v, u] - F[v, u]))$; $a := 1$
 end if
 end for
 end if
 end for
 if $S[t]$ **then**
 $x := t$ { текущий узел аугментальной цепи }
 $\delta := P[t].\delta$ { величина изменения потока }
 while $x \neq s$ **do**
 if $P[x].s = +$ **then**
 $F[P[x].n, x] := F[P[x].n, x] + \delta$ { увеличение потока }
 else
 $F[x, P[x].n] := F[x, P[x].n] - \delta$ { увеличение потока }
 end if
 $x := P[x].n$ { предыдущий узел аугментальной цепи }
 end while
 goto M
 end if
until $a = 0$

Рис. 10: Псевдокод алгоритма Форда — Фалкерсона, часть 1

end for
for $u \in \Gamma^{-1}(v)$ **do**
 if $S[u] = 0 \ \& \ F[u, v] > 0$ **then**
 $S[u] := 1; P[u] := (-, v, \min(P[v].\delta, F[u, v]))$; $a := 1$
 end if
end for
 $N[v] := 1$ { узел v отмечен }
end if
end for
if $S[t]$ **then**
 $x := t$ { текущий узел аугментальной цепи }
 $\delta := P[t].\delta$ { величина изменения потока }
 while $x \neq s$ **do**
 if $P[x].s = +$ **then**
 $F[P[x].n, x] := F[P[x].n, x] + \delta$ { увеличение потока }
 else
 $F[x, P[x].n] := F[x, P[x].n] - \delta$ { увеличение потока }
 end if
 $x := P[x].n$ { предыдущий узел аугментальной цепи }
 end while
 goto M
end if
until $a = 0$

Рис. 11: Псевдокод алгоритма Форда — Фалкерсона, часть 2

Последовательное увеличение потока на примере сети показано на рис. 12.

Рассмотрим сеть S , и пусть f_0 — нулевой поток.

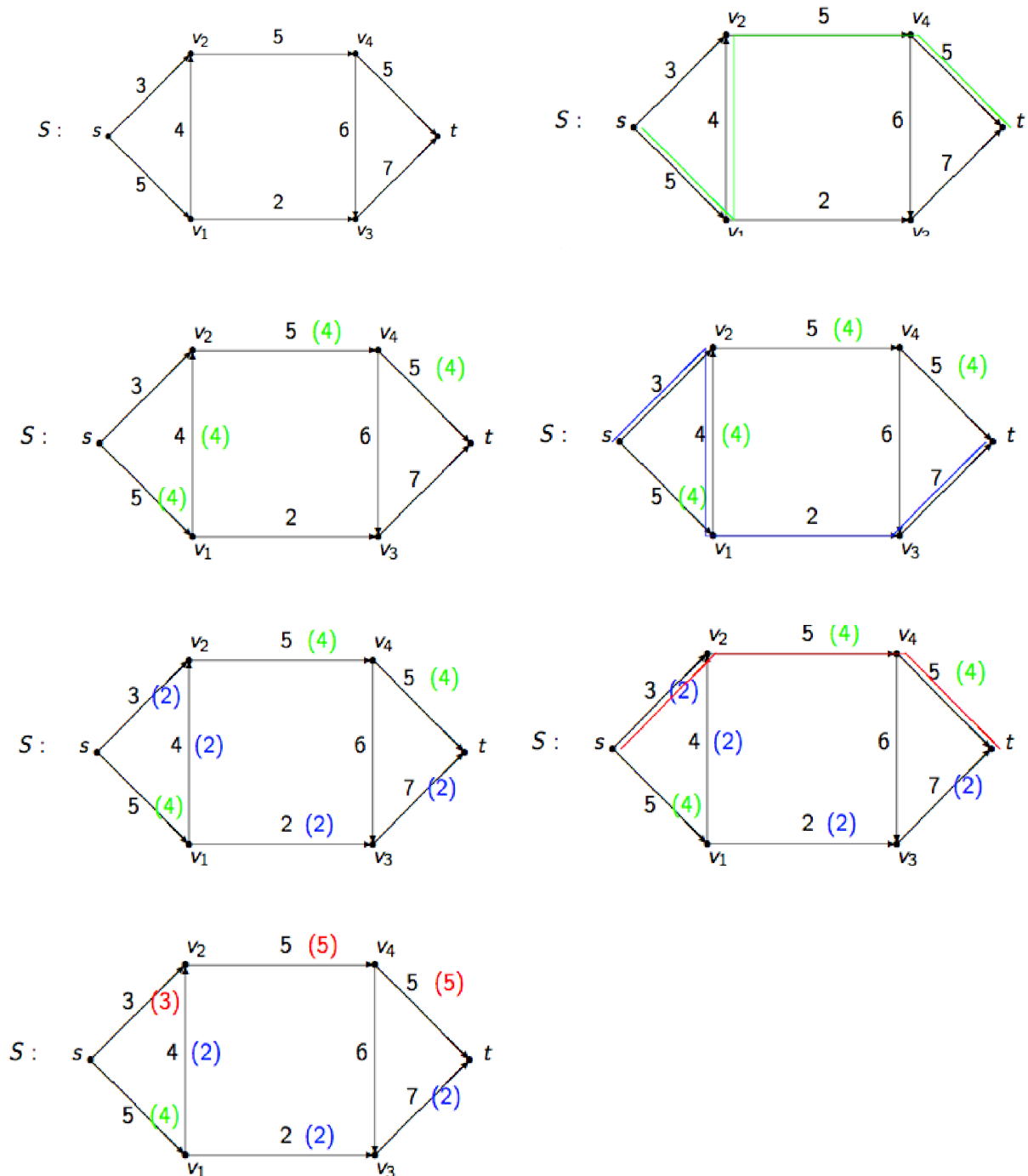


Рис. 12: Последовательное построение максимального потока

Пусть каждой дуге $e \in E$ также поставлена в соответствие стоимость $a(e)$ единицы проходящего по ней потока. Тогда стоимость потока f определяется суммой

$$C(f) = \sum_{e \in E} a(e)f(e).$$

В работе требуется найти поток минимальной стоимости заданной величины

$$F_{\text{req}} = \left\lfloor \frac{2}{3} F_{\text{max}} \right\rfloor,$$

где F_{max} — величина максимального потока, найденная алгоритмом Форда — Фалкерсона. Если $F_{\text{max}} = 1$, принимается $F_{\text{req}} = 1$, чтобы заданная величина потока оставалась положительной.

Поток минимальной стоимости строится в полученной ранее сети. На каждой итерации модифицированным алгоритмом Дейкстры находится путь наименьшей стоимости от истока к стоку. После этого величина потока увеличивается на максимально допустимое значение, определяемое пропускными способностями рёбер найденного пути. Вычисления продолжаются до достижения F_{req} либо до отсутствия такого пути.

2.4 Остовы, код Прюфера и независимые множества рёбер

Граф без циклов называется ациклическим, или лесом. Связный ациклический граф называется свободным деревом.

Пусть $G = (V, E)$ — граф. Остовным подграфом графа G называется подграф, содержащий все вершины графа G . Остовный подграф, являющийся деревом, называется остовом. Поскольку вершины остова совпадают с вершинами исходного графа, остов однозначно задаётся множеством своих рёбер. Несвязный граф не имеет остова, а связный граф может иметь несколько остовов.

Матрицей Кирхгофа графа G называется матрица $B = (b_{ij})$, диагональные элементы которой равны степеням соответствующих вершин, а остальные элементы равны -1 для смежных вершин и нулю в противном случае. По теореме Кирхгофа число остовов связного графа равно алгебраическому дополнению любого элемента матрицы B . Пример вычисления приведён на рис. 13.

- **Теорема Кирхгофа.** Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$.
- **Н.В.** Все алгебраические дополнения матрицы Кирхгофа равны между собой.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 2 & -1 & 0 & -1 & 0 & 0 \\ -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & -1 & 2 & 0 & -1 & 0 \\ -1 & -1 & 0 & 3 & -1 & 0 \\ 0 & -1 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_{23} = (-1)^{2+3} \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 1 \end{vmatrix} = (-1)^5 \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{vmatrix} = (-1)^{5+5+5} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & -1 & 0 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 3 \end{vmatrix} =$$

$$= (-1)^{15} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 3 & -1 \\ 0 & -4 & -1 & 3 \end{vmatrix} = (-1)^{16+4+2} \begin{vmatrix} 2 & -1 & -1 \\ -1 & 0 & 3 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{22} \begin{vmatrix} 2 & -1 & 5 \\ -1 & 0 & 0 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{26} \begin{vmatrix} -1 & 5 \\ -4 & -1 \end{vmatrix} = 1 + 20 = 21$$

Рис. 13: Вычисление числа остовов по теореме Кирхгофа

Минимальным остовом взвешенного графа называется остов с наименьшей суммой весов рёбер. Алгоритм Краскала рассматривает рёбра в порядке возрастания их весов. В начале каждая вершина образует отдельное множество. Очередное ребро добавляется в остов, если его концы принадлежат разным множествам, после чего эти множества объединяются. Такая проверка не допускает образования циклов. После выбора $|V| - 1$ ребра получается минимальный остов. Псевдокод алгоритма и пример последовательного построения остова приведены на рис. 14.

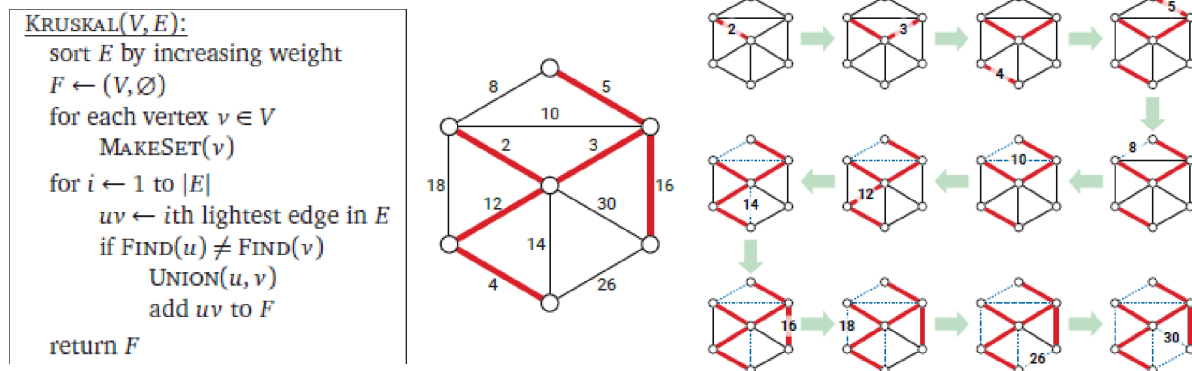


Рис. 14: Алгоритм Краскала и пример его работы

Множество рёбер называется независимым, или паросочетанием, если никакие два его ребра не смежны. Наибольшее число рёбер в независимом множестве рёбер называется рёберным числом независимости и обозначается β_1 .

Вершина, не инцидентная ни одному ребру паросочетания, называется свободной. Увеличивающий путь начинается и заканчивается в свободных вершинах, а его рёбра попеременно не принадлежат и принадлежат паросочетанию. Замена входящих в паросочетание рёбер такого пути на не входящие увеличивает число выбранных рёбер на один.

Алгоритм Эдмондса ищет увеличивающие пути обходом в ширину. Если в дереве поиска обнаруживается цикл нечётной длины, он временно сжимается в одну вершину, после чего поиск продолжается. При нахождении увеличивающего пути сжатые циклы раскрываются, а паросочетание увеличивается. Если увеличивающих путей больше нет, полученное паросочетание является максимальным. Псевдокод алгоритма приведён на рис. 15.

Подробное описание и интерактивная визуализация алгоритма приведены в материале Технического университета Мюнхена “[Edmonds’s Blossom Algorithm](#)”.

Algorithm status

prev
next
fast forward

Explanation
Pseudocode

```

BEGIN
  WHILE F != ∅ DO      (*set of free nodes F*)
    pick r ∈ F
    queue.push(r)      (*BFS queue*)
    T ← ∅              (*BFS tree T*)
    T.add(r)
    WHILE queue != ∅
      v ← queue.pop()
      FOR ALL neighbors w of v DO
        IF w ∉ T AND w matched THEN
          T.add(w)
          T.add(mate(w))
          queue.push(mate(w))
        ELSE IF w ∈ T AND even-length
          cycle detected THEN
          CONTINUE
        ELSE IF w ∈ T AND odd-length
          cycle detected THEN
          contract cycle
        ELSE IF w ∈ F THEN
          expand all contracted nodes
          reconstruct augmenting path
          invert augmenting path
      END
    END
  END

```

Рис. 15: Псевдокод алгоритма Эдмондса

2.5 Эйлеровы циклы и фундаментальная система разрезов

Если граф имеет цикл (не обязательно простой), содержащий все рёбра графа, то такой цикл называется эйлеровым циклом, а граф — эйлеровым графом. *Эйлеровым может быть только связный граф.*

Для приведения графа к эйлерову виду сначала находятся вершины нечётной степени. Из них выбираются две вершины u и v . Если они не смежны, ребро (u, v) добавляется. Если ребро существует, оно удаляется только при наличии обходного пути между u и v , что сохраняет связность графа. После каждой операции чётность степеней u и v меняется, поэтому число вершин нечётной степени уменьшается на два. Поскольку таких вершин всегда чётное число, процесс завершается за конечное число шагов.

Для построения эйлерова цикла применяется алгоритм Флёрри. Начальная вершина помещается в стек. Из текущей вершины выбирается соседняя вершина, после чего соединяющее их ребро удаляется, а выбранная вершина добавляется в стек. Переходы продолжаются, пока у текущей вершины не останется соседей. Такая вершина добавляется в результат и удаляется из стека, после чего обход продолжается из предыдущей вершины. После опустошения стека получается эйлеров цикл. Псевдокод алгоритма приведён на рис. 16.

Вход: эйлеров граф $G(V, E)$, заданный списками смежности ($\Gamma[v]$ — список вершин, смежных с вершиной v).

Выход: последовательность вершин эйлерова цикла.

```
 $S := \emptyset$  { стек для хранения вершин }  
select  $v \in V$  { произвольная вершина }  
 $v \rightarrow S$  { положить  $v$  в стек  $S$  }  
while  $S \neq \emptyset$  do  
   $v := \text{top } S$  {  $v$  — верхний элемент стека }  
  if  $\Gamma[v] = \emptyset$  then  
     $v \leftarrow S$ ; yield  $v$  { очередная вершина эйлерова цикла }  
  else  
    select  $u \in \Gamma[v]$  { взять первую вершину из списка смежности }  
     $u \rightarrow S$  { положить  $u$  в стек }  
     $\Gamma[v] := \Gamma[v] - u$ ;  $\Gamma[u] := \Gamma[u] - v$  { удалить ребро  $(v, u)$  }  
  end if  
end while
```

Если реализовать поиск ребер инцидентных вершине и удаление ребер за $O(1)$, то алгоритм будет работать за $O(p)$.

Рис. 16: Псевдокод алгоритма Флёрри

3 Результаты работы

Для сравнения было выполнено 100 запусков на случайных графах из 10 вершин. В каждом запуске оба алгоритма работали на одном графе. Количество итераций в отдельных запусках показано на рис. 17.

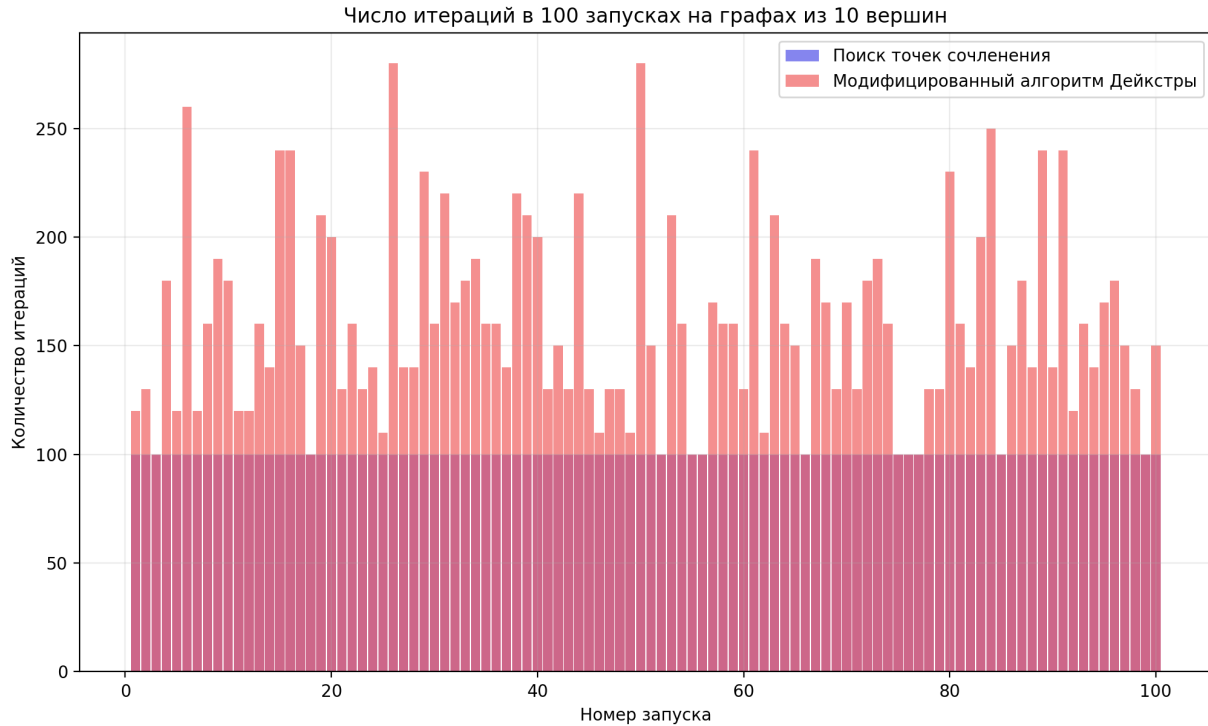


Рис. 17: Количество итераций в каждом запуске

Распределение полученных значений приведено на рис. 18.

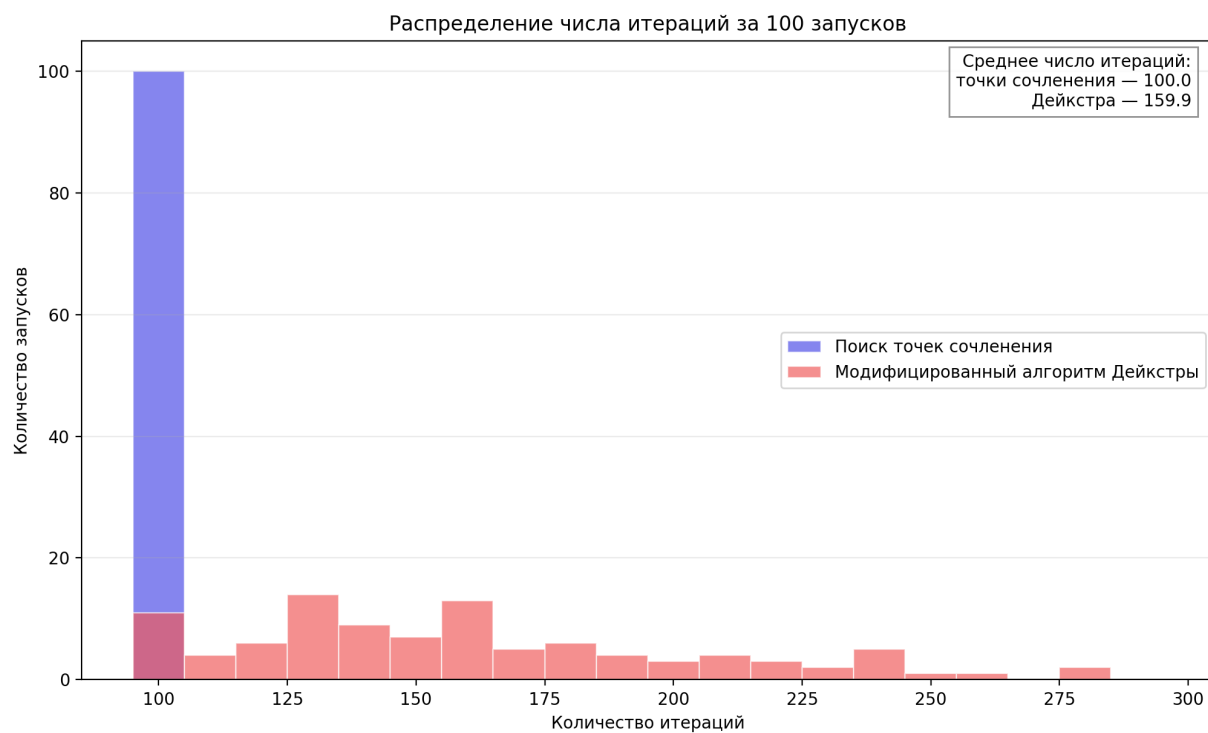


Рис. 18: Распределение количества итераций за 100 запусков

Поиск точек сочленения во всех случаях выполнил 100 итераций. В 89 запусках поиск точек сочленения потребовал меньше итераций, а в 11 запусках значения совпали. Следовательно, на проведённой серии запусков поиск точек сочленения оказался эффективнее по количеству итераций.